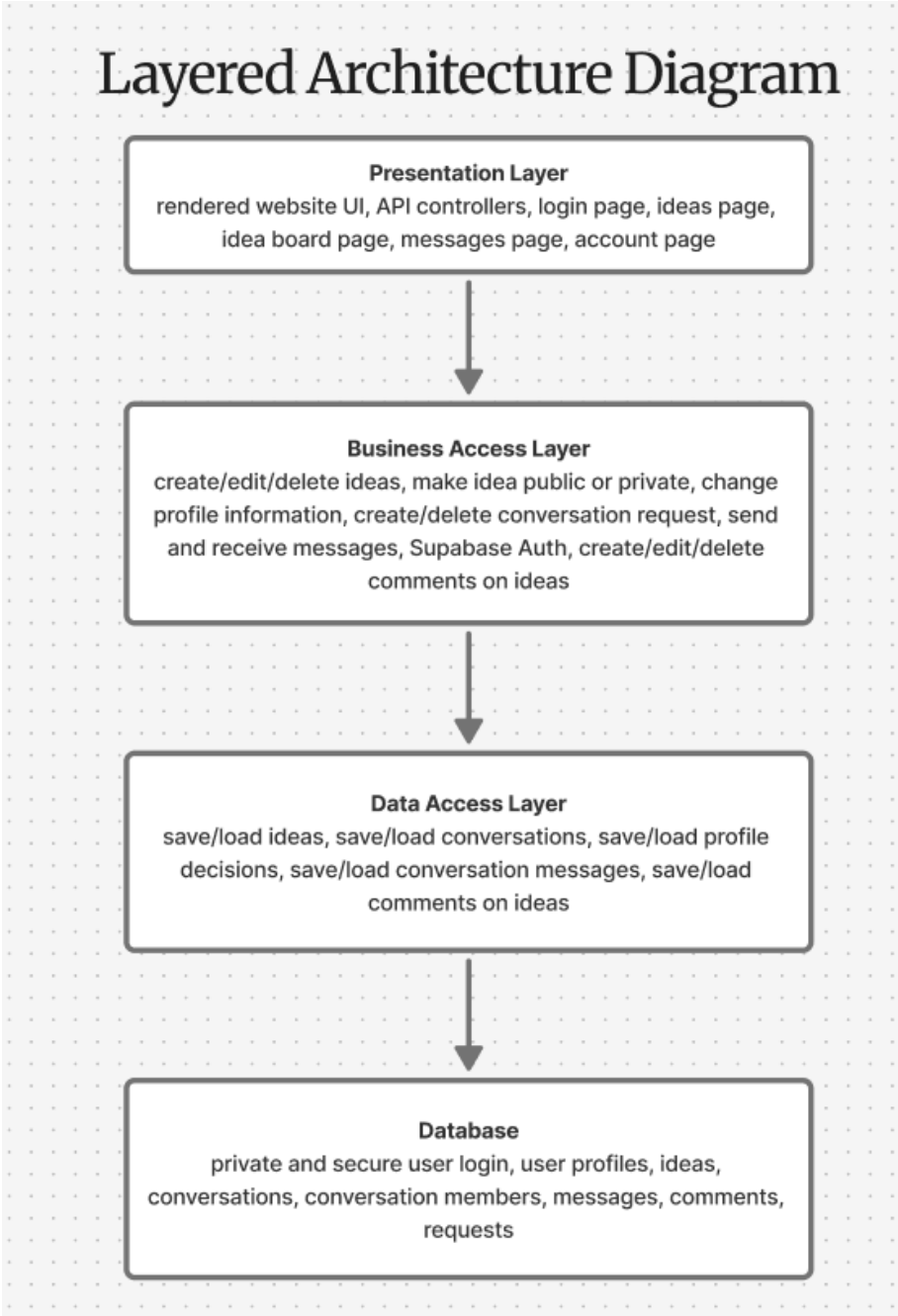
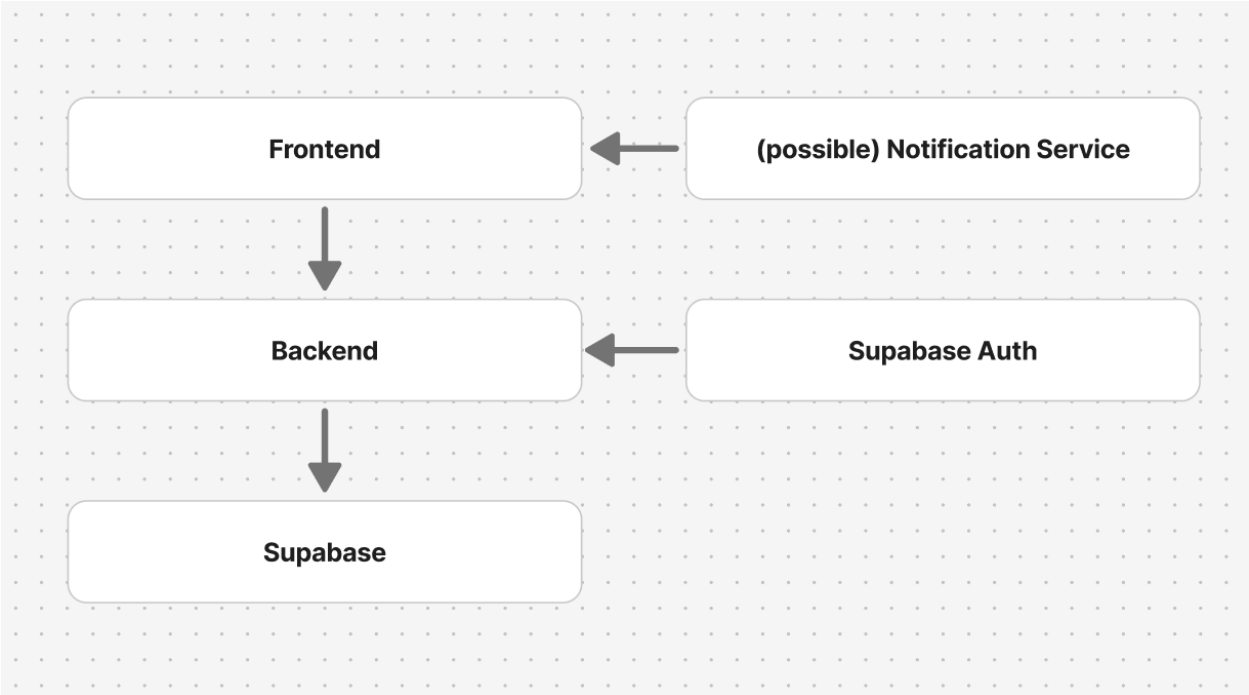
1. Proposed Architecture

We believe our primary architecture pattern to be Layered Architecture. While our idea component more closely aligns with Pipe-and-Filter, our application as a whole and the different services it provides leans more toward Layer Architecture due to the clear responsibilities each layer has and how successfully we have implemented that along with the maintainability of our code, only having to change one layer at a time.



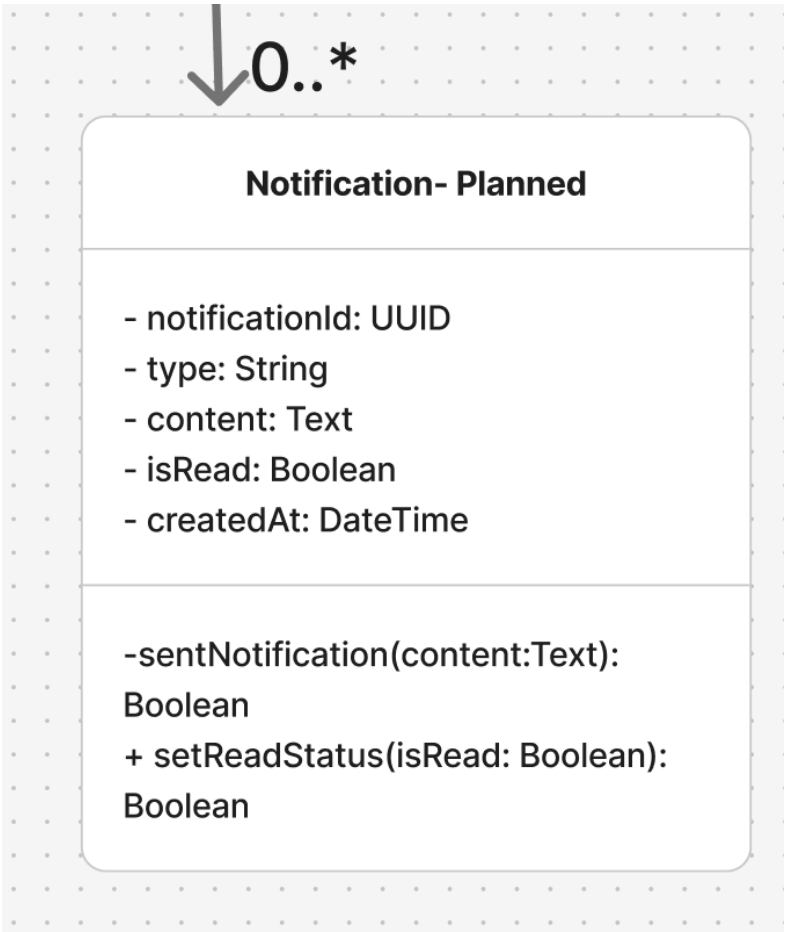
2. *High-Level Design Diagram*

This was our interpretation of what was asked with a high-level design diagram from the rubric as well as from the Professor.



3. *Design Patterns*

After viewing the options in class, we decided the Observer pattern most aligned with our intentions and ideas for implementation. For our messaging functionality, we wanted the user to receive a notification every time they received a new message or a message request. Having this separate is better than one long piece of code in one file because it improves organization, scalability, and maintainability. An outside notification service or API *may or may not* be implemented later, but for now we plan to make it ourselves.



4. Design Decisions and Tradeoffs

a. The first decision we chose was to use Supabase Auth rather than creating our own authentication system for user login. Between those two options, we went with Supabase Auth due to the fact that it is easy to set up and has all of the needed options built in for us. Creating our own from scratch takes hundreds of hours on average and for the scope of this class and project, it seemed much smarter to go with Supabase Auth.

b. The second decision we chose was to create our own messaging functionality rather than using a premade messaging API. Between those two options, we chose to make our own due to our want to tailor it to our needs and keep all of our services within Supabase rather than juggling additional external vendors and their possible fees.